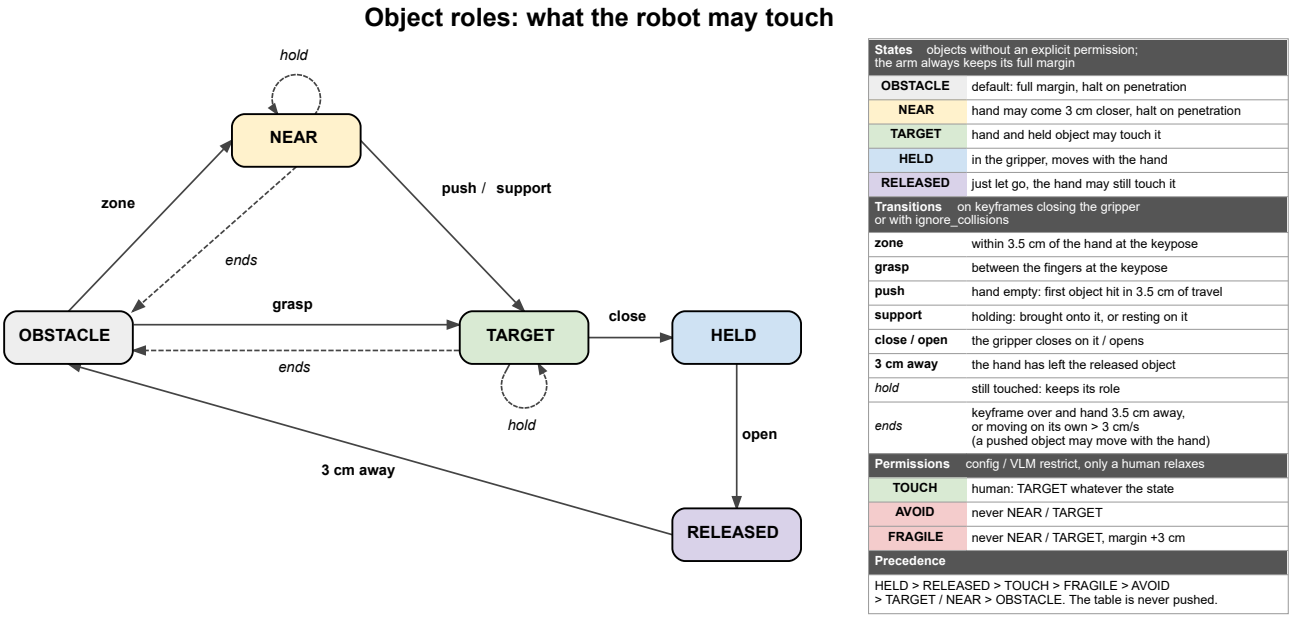
Speech per le slide "Object roles" e "Keyframe execution"

Durata totale: circa 4 minuti. Tra parentesi quadre: cosa indicare sulla slide. Le slide sono in inglese, il discorso in italiano: le parole sulle frecce si leggono così come sono.

Slide 1 — Object roles: what the robot may touch (2,5–3 minuti)



States	objects without an explicit permission; the arm always keeps its full margin
OBSTACLE	default: full margin, halt on penetration
NEAR	hand may come 3 cm closer, halt on penetration
TARGET	hand and held object may touch it
HELD	in the gripper, moves with the hand
RELEASED	just let go, the hand may still touch it
Transitions	on keyframes closing the gripper or with ignore_collisions
zone	within 3.5 cm of the hand at the keypose
grasp	between the fingers at the keypose
push	hand empty: first object hit in 3.5 cm of travel
support	holding: brought onto it, or resting on it
close / open	the gripper closes on it / opens
3 cm away	the hand has left the released object
hold	still touched: keeps its role
ends	keyframe over and hand 3.5 cm away, or moving on its own > 3 cm/s (a pushed object may move with the hand)
Permissions	config / VLM restrict, only a human relaxes
TOUCH	human: TARGET whatever the state
AVOID	never NEAR / TARGET
FRAGILE	never NEAR / TARGET, margin +3 cm
Precedence	
HELD > RELEASED > TOUCH > FRAGILE > AVOID > TARGET / NEAR > OBSTACLE. The table is never pushed.	

Perché serve (30 s)

[indica il titolo]

> "Nella DreMa originale il modello del mondo serve a generare dati: il robot viene poi mosso da un pianificatore che, nei movimenti di contatto, ignora le collisioni con tutto. Nel nostro sistema il modello del mondo è dentro il ciclo di controllo, con ostacoli che si muovono. Non possiamo ignorare tutto: dobbiamo decidere, oggetto per oggetto, cosa il robot può toccare. Questo automa è quella decisione, e viene ricalcolato a ogni passo di controllo."

Lo stato di partenza (15 s)

[indica OBSTACLE]

> "Ogni oggetto nasce come ostacolo: mano e braccio gli stanno lontani con il margine pieno, e se nel twin compare una compenetrazione scatta l'arresto di sicurezza."

Come un oggetto diventa toccabile (60 s)

[segui la freccia zone verso NEAR]

> "Quando il keyframe prevede un contatto — la pinza si chiude, oppure PerAct segnala `ignore_collisions` — guardiamo dove sarà la mano alla posa finale. Gli oggetti vicini, la zona di contatto, diventano NEAR: la mano può avvicinarsi di più, ma non entrarci."

[indica grasp e push / support]

> "Un oggetto diventa TARGET, cioè toccabile dalla mano, solo in tre casi, che sono i tre contatti previsti di un compito di manipolazione: la **presa**, se è tra le dita alla posa finale; la **spinta**, se con la mano vuota è il primo oggetto che la mano incontra all'inizio del movimento — il blocco di slide_block; l'**appoggio**, se tenendo un oggetto ci si avvicina o ci si appoggia sopra — il barattolo sotto il coperchio in close_jar."

[segui close, open, 3 cm away]

> "Quando la pinza si chiude l'oggetto diventa HELD: nel twin si muove con la mano ed è controllato contro gli altri. Quando si apre diventa RELEASED, e torna ostacolo appena la mano si è allontanata."

Quando un ruolo finisce (30 s)

[indica le frecce tratteggiate ends e gli anelli hold]

> "Un ruolo concesso dalla policy dura solo per quel keyframe: alla fine torna ostacolo. Con due eccezioni. La prima è *hold*: finché la mano o l'oggetto tenuto lo stanno ancora toccando, il ruolo resta, altrimenti il robot si bloccherebbe per un arresto proprio mentre si stacca. La seconda: se l'oggetto si muove per conto suo — il tunnel — il permesso viene revocato subito."

Autorità e sicurezza (30 s)

[indica nella legenda Permissions e Precedence]

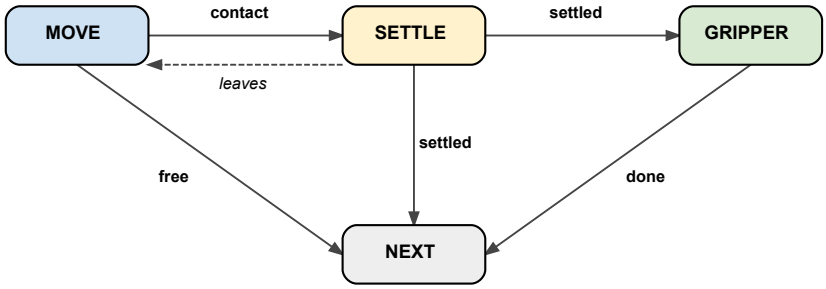
> "Sopra tutto ci sono i permessi espliciti. Configurazione e VLM possono solo restringere: evita, fragile. Solo un umano può allargare un permesso. Un oggetto AVOID o FRAGILE non diventa mai toccabile per decisione automatica. E il braccio mantiene sempre il margine pieno: i permessi riguardano soltanto la mano."

Chiusura (15 s)

> "In sintesi: ogni regola è locale — solo la mano —, temporanea — solo quel keyframe — e nasce da due fonti: l'intento del keyframe e la geometria del twin. Ognuna si può disattivare singolarmente."

Slide 2 — Keyframe execution: when a keyframe counts as reached (1–1,5 minuti)

Keyframe execution: when a keyframe counts as reached



States	client sequencer; the server MPPI decides how the arm moves
MOVE	the MPPI tracks the keypose (approach cone on grasps)
SETTLE	wait until the arm stops closing in
GRIPPER	close / open, wait for the fingers
NEXT	the next keyframe becomes active
Transitions	
free	free keyframe: within 2 cm
contact	grasp / release: within 1 cm; approach pose before them: within 2 cm
settled	within 3 mm, or less than 1 mm closer over 10 steps (0.5 s), or 60 steps: the gripper acts on a grasp / release, an approach pose goes straight to NEXT
leaves	the arm leaves the tolerance while settling
done	the fingers have stopped
Note	
Approach poses are known for demo keyframes only: a policy gives one keyframe at a time.	

Il problema (30 s)

> "PerAct fornisce pose chiave. In RLBench un pianificatore le raggiunge esattamente. Il nostro MPPI è un controllore reattivo e arriva entro una tolleranza. Abbiamo misurato che, chiudendo la pinza al primo istante sotto il centimetro, l'oggetto veniva preso quasi un centimetro fuori posto, e all'appoggio finiva sul bordo del barattolo invece che sopra."

La soluzione (40 s)

[segui MOVE → contact → SETTLE → settled → GRIPPER → done → NEXT]

> "Per i keyframe di presa e rilascio, e per la posa che li precede, entrare nella tolleranza non basta: nello stato SETTLE aspettiamo che il braccio smetta di avvicinarsi. Solo allora si aziona la pinza, o si passa oltre. I keyframe liberi restano veloci *[indica free]*. Se il braccio non riesce ad avvicinarsi di più, dopo poco si procede comunque: il robot non resta mai fermo."

Risultato e limite (40 s)

> "Con questa regola l'errore al momento della presa è sceso da circa 0,9 a 0,3 centimetri, e così anche al rilascio. In close_jar, prima, il robot arrivava al rilascio a 1–1,4 centimetri dalla posa: il coperchio si fermava sul bordo e il task falliva due volte su cinque. Ora arriva a 0,3 centimetri e il coperchio entra. Il prezzo è qualche decimo di secondo di attesa per ogni presa e rilascio. Il limite: la posa che precede una presa la conosciamo solo con le demo; con PerAct, che predice un keyframe alla volta, è un lavoro futuro."

Numeri preliminari (diagnostici del 30/10, pochi episodi): da sostituire con quelli della suite completa sul server. Prima: presa a 0,7–0,9 cm, close_jar 3/5 con rilascio bloccato a 1,1–1,4 cm. Dopo: presa e rilascio a 0,2–0,5 cm, close_jar riuscito negli episodi completati.

Se chiedono cosa non funziona ancora (20 s)

> "I fallimenti che restano vengono soprattutto dalla percezione: oggetti ricostruiti più grandi o spostati di uno o due centimetri, che nel twin si compenetrano con la mano o tra loro, o oggetti sottili non visti. Il sistema non li compensa — sarebbe nascondere l'errore — ma non deve bloccarsi per colpa loro: è il motivo della regola *hold*. La correzione vera è compito del modulo di percezione."

Esempio da raccontare se c'è tempo: close_jar sul diagramma

1. Il coperchio è nella zona di presa: OBSTACLE → *grasp* → TARGET → *close* → HELD.
2. Sopra il barattolo, il keyframe di appoggio: il barattolo giusto diventa TARGET per *support*; l'altro barattolo resta NEAR.
3. *open*: il coperchio diventa RELEASED, poi *3 cm away* e torna OBSTACLE; il barattolo resta TARGET per *hold* finché la mano lo tocca, poi *ends*.

Domande probabili

- **Perché non far toccare tutto durante un movimento di contatto, come il pianificatore originale?** Perché il tunnel si muove anche durante il contatto: va evitato sempre.
- **Chi decide i permessi?** Solo l'umano li allarga; configurazione e VLM possono solo restringere.
- **E se la percezione sbaglia?** Gli errori di percezione appaiono come compenetrazioni nel twin: la regola *hold* evita i blocchi, ma la correzione vera spetta al modulo di percezione.
- **I numeri (3,5 cm, 1 cm, 3 cm/s) da dove vengono?** Sono soglie della configurazione, ognuna con il suo interruttore; 3,5 cm è il raggio in cui la mano già "sente" il costo di collisione.

Domande tecniche (dettagli da sapere)

Come agiscono i ruoli sul controllore

- I ruoli non cambiano l'obiettivo dell'MPPI: cambiano **quali coppie (parte del robot, oggetto) entrano nel costo di collisione e nell'arresto di sicurezza**, e con che margine. OBSTACLE: margine pieno. NEAR: la mano ha un margine ridotto di 3 cm, ma la compenetrazione fa comunque scattare l'arresto. TARGET: la coppia mano–oggetto (e oggetto tenuto–oggetto) è esclusa sia dal costo sia dall'arresto. Il braccio ha sempre il margine pieno.
- Il costo di collisione è calcolato nel twin (PyBullet o MuJoCo, stesse regole) per ogni link e ogni rollout, con gli ostacoli mossi nella posa prevista dal predittore lungo l'orizzonte.
- **Arresto di sicurezza**: se nel twin la distanza minima tra robot e un oggetto non esentato scende sotto zero (compenetrazione), il controllore manda velocità nulle. Le coppie esentate (TARGET) non contano.

Geometria della mano

- La "mano" è un ingombro fatto di tre box nel sistema del TCP: palmo, dita, spazio tra le dita (unione delle mani dei due twin). La **zona** è la distanza (entro 3,5 cm) tra questi box, messi alla posa del keyframe, e la superficie degli oggetti; la **presa** usa solo il box tra le dita (entro 1 cm).
- Perché 3,5 cm: è `sigma_2`, la distanza a cui il costo di collisione dell'MPPI inizia a farsi sentire. Un oggetto più lontano non influenza comunque la mano.

Spinta

- All'inizio del keyframe facciamo scorrere l'ingombro della mano in linea retta verso la posa finale, a passi di 5 mm, per i primi 3,5 cm. Il primo oggetto che la mano tocca (entro 5 mm) **e a cui si sta avvicinando** (5 mm più vicino che all'inizio) diventa TARGET. Un oggetto di lato non conta: la mano gli scorre accanto senza avvicinarsi.
- "Ma l'MPPI non va in linea retta": è un'approssimazione per decidere *quale* oggetto è previsto. Se il robot devia e tocca altro, quell'oggetto è ancora ostacolo: costo o arresto, cioè un fallimento sicuro.
- Se il primo oggetto incontrato è AVOID, FRAGILE o in movimento, non si promuove niente, nemmeno ciò che sta dietro.
- Un oggetto spinto si muove: non viene revocato finché si muove **con la mano** (velocità relativa oggetto–mano sotto 3 cm/s). La velocità della mano viene dalla cinematica e dalle velocità dei giunti; quella dell'oggetto dal predittore (filtro di Kalman sul tracking).

Appoggio

- Tenendo un oggetto, lo mettiamo alla posa finale e guardiamo gli oggetti entro 3,5 cm. È appoggio un oggetto a cui il keyframe **avvicina** l'oggetto tenuto (almeno 1 cm più vicino che all'inizio) o su cui l'oggetto **resta appoggiato** (entro 1 cm all'inizio e alla fine: avvitare, trascinare). Diventano TARGET il più vicino più quelli a contatto (il fondo e la parete di uno scaffale).
- In un sollevamento ci si allontana da tutto: nessun appoggio.
- L'oggetto tenuto può toccare **solo il suo appoggio**: gli altri oggetti vicini (NEAR) restano protetti sia dalla mano sia dall'oggetto tenuto. In close_jar il coperchio può poggiare sul barattolo giusto, non urtare l'altro.

Presa e rilascio nel twin

- La presa viene riconosciuta dal movimento delle dita: quando si fermano chiudendosi, l'oggetto toccabile più vicino al TCP (entro 4 cm) viene **agganciato rigidamente** alla mano nel twin, con l'offset misurato in quel momento. Da lì segue la mano in ogni rollout e la percezione (spesso coperta dalle dita) viene ignorata. Se le dita si chiudono quasi del tutto, la presa è "a vuoto" e non si aggancia niente.
- Al rilascio l'oggetto viene sganciato e la percezione torna a comandare; resta RELEASED finché il TCP non è a 3 cm.

Hold e revoca

- Al cambio di keyframe, un oggetto che la mano o l'oggetto tenuto toccano ancora (entro 3,5 cm) mantiene il suo ruolo, anche se la nuova zona gli darebbe un ruolo più basso. Esempio reale: in shape_sorter un oggetto vicino era TARGET durante la presa; nel sollevamento la zona lo rendeva NEAR e, per un errore di percezione di pochi mm, l'arresto bloccava il robot. Con *hold* resta TARGET finché la mano non si stacca.
- Un oggetto che si muove per conto suo oltre 3 cm/s perde il ruolo fino al keyframe successivo: un ostacolo mobile non è mai toccabile.

Esecuzione dei keyframe

- Tolleranze: keyframe libero 2 cm, presa / rilascio 1 cm, posa che li precede 2 cm più assestamento.
- Assestato: entro 3 mm, oppure meno di 1 mm di avvicinamento negli ultimi 10 passi (0,5 s a 20 Hz), oppure dopo 60 passi (3 s) dentro la tolleranza: il robot non resta mai fermo.
- Perché una tolleranza e non la posa esatta: l'MPPI è reattivo e converge in modo asintotico; con ostacoli in movimento non può "chiudere" su una traiettoria fissa come un pianificatore.
- Il cono di avvicinamento (*approach cone*) vale solo sulle prese: la pinza scende lungo il proprio asse. Non lo usiamo sul rilascio perché non è generico (appendere, inserire di lato) e, misurato, non avrebbe cambiato i casi visti.

PerAct e `ignore_collisions`

- In PerAct il flag accompagna ogni keyframe e si riferisce al **movimento verso quella posa**; dalle demo si prende quello del passo precedente al keyframe. Per noi significa "in questo movimento è previsto un contatto": è la condizione che attiva zona, spinta e appoggio.

Generalità e verifica

- Le regole usano solo l'intento del keyframe (pinza, `ignore_collisions`) e la geometria del twin: nessuna regola specifica per un task. Test unitari su entrambi i twin per ogni regola, e test su task RLBench diversi (pick_and_lift, stack_blocks, close_jar, shape_sorter, insert_peg, slide_block, pick_up_cup) più i due task dinamici.
- Il task con il tunnel in movimento (test_1) dà le stesse metriche di prima: le regole nuove non lo toccano.

Differenze con M3P2I e con la DreMa originale

- **DreMa originale**: il modello del mondo genera dati di addestramento offline; l'esecuzione usa un pianificatore esatto in scena statica. Noi mettiamo il modello del mondo nel ciclo di controllo, con ostacoli dinamici.
- **M3P2I**: simula la fisica degli oggetti in IsaacGym su GPU, con modelli noti e costi scritti per ogni abilità. Noi usiamo un twin ricostruito dalla percezione e i keyframe di una policy appresa; la simulazione della risposta degli oggetti spinti (livello 2) è il passo successivo, e l'obiettivo sull'oggetto tenuto ne sarebbe il primo mattone.

Chi fa cosa nell'architettura

- **Client** (ambiente o robot, come il ciclo di PerAct): fornisce i keyframe e decide quando un keyframe è raggiunto e quando azionare la pinza (assestamento).
- **Server**: l'MPPI decide *come* muoversi (costi, cono di avvicinamento); il registro dei ruoli decide *cosa* si può toccare; il twin ricostruito fornisce la geometria.
- **Percezione**: l'accuratezza della scena. I suoi errori non vengono compensati nei ruoli o nel controllo.

Livelli del modello fisico del mondo

- **Livello 0**: evitare, permettere il contatto, tenere in mano in modo rigido.
- **Livello 1** (dove siamo): contatto passivo, cioè toccare e spingere ciò che il keyframe prevede, senza predire dove finisce l'oggetto. È quello che serve a PerAct, che codifica la spinta con posa chiave più `ignore_collisions`.
- **Livello 2**: i rollout simulano la risposta dell'oggetto spinto e il costo guarda dove finisce (come M3P2I).
- **Livello 3**: decidere da soli di spostare un ostacolo per raggiungere il keyframe.
- I ruoli restano anche nei livelli successivi: la fisica dice *cosa succede* toccando, i ruoli dicono *se si può* toccare.

Limiti da dire apertamente

- Errori di percezione di 1–2 cm (mesh gonfiate, oggetti sottili non visti) causano gran parte dei fallimenti residui.
- La spinta di un oggetto contro un altro non è prevista: i rollout non simulano dove va l'oggetto spinto.
- I permessi valgono per l'oggetto intero (un appoggio grande è toccabile ovunque durante quel keyframe).
- Con PerAct reale la posa che precede la presa non è nota in anticipo.